

ASSIGNMENT-11

TASK-1:

Prompt:

Implement a Stack class in Python with the following operations: push(), pop(), peek(), and is_empty(). Generate code skeleton with docstrings also Test stack operations using sample data. Use optimizations or alternative implementations (e.g., using collections.deque) if needed.

Code and Output:

```
lab 11.py > ...
1  from collections import deque
2
3  class Stack:
4      """
5      A simple Stack implementation using collections.deque for efficient appends and pops.
6      """
7      def __init__(self):
8          """
9          Initialize an empty stack.
10         """
11         self._container = deque()
12
13     def push(self, item):
14         """
15         Push an item onto the stack.
16
17         Args:
18             item: The item to be pushed.
19         """
20         self._container.append(item)
21
22     def pop(self):
23         """
24         Pop the top item off the stack and return it.
25
26         Returns:
27             The item popped from the stack.
28
29         Raises:
30             IndexError: If the stack is empty.
31         """
32         if self.is_empty():
33             raise IndexError("pop from empty stack")
34         return self._container.pop()
35
36     def peek(self):
37         """
38         Return the top item of the stack without removing it.
39
40         Returns:
41             The top item of the stack.
42
43         Raises:
44             IndexError: If the stack is empty.
45         """
```

```

46         if self.is_empty():
47             raise IndexError("peek from empty stack")
48         return self._container[-1]
49
50     def is_empty(self):
51         """
52         Check whether the stack is empty.
53
54         Returns:
55             True if the stack is empty, False otherwise.
56         """
57         return len(self._container) == 0
58
59
60 if __name__ == "__main__":
61     # Sample test operations for the Stack class
62     stack = Stack()
63
64     # Test pushing items
65     print("Pushing 1, 2, 3 onto the stack.")
66     stack.push(1)
67     stack.push(2)
68     stack.push(3)
69
70     # Test peeking at the top item
71     print("Current top item (peek):", stack.peek())
72
73     # Test popping item
74     print("Popped item:", stack.pop())
75     print("New top item after pop (peek):", stack.peek())
76
77     # Test popping remaining items
78     print("Popped item:", stack.pop())
79     print("Popped item:", stack.pop())
80
81     # Test is_empty() method
82     print("Is stack empty?", stack.is_empty())
83
84     # Trying to pop from an empty stack to see error handling (uncomment to test)
85     # print("Attempting to pop from empty stack:")
86     # print(stack.pop())

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

New top item after pop (peek): 2
Popped item: 2
Popped item: 1
Is stack empty? True

```

Code Explanation:

The file defines a `Stack` class using Python's `collections.deque` for efficient stack operations. Here's a breakdown of the code:

- **Imports:**

The `deque` from the `collections` module is imported because it provides fast appends and pops from the end of the deque, making it a good choice for a stack.

- **Stack Class:**

The `Stack` class encapsulates the behavior of a standard stack with the following methods:

- `__init__(self)` :
Initializes the stack by creating an empty `deque` as its underlying container.
- `push(self, item)` :
Adds an item to the top of the stack by appending it to the deque.
- `pop(self)` :
Removes and returns the item at the top of the stack. Before doing that, it checks if the stack is empty and raises an `IndexError` if it is.
- `peek(self)` :
Returns the item at the top of the stack without removing it. Similar to `pop()`, it checks if the stack is empty and raises an `IndexError` if so.
- `is_empty(self)` :
Returns `True` if the stack contains no items; otherwise, it returns `False`.

- **Testing the Stack:**

In the `if __name__ == "__main__":` block, the code demonstrates the usage of the `Stack` class:

- It creates an instance of `Stack`.
- It pushes three values (1, 2, 3) onto the stack.
- It uses `peek()` to show the top item.
- It pops an item to show the removal of the top element and again uses `peek()` to show the new top.
- It then pops the remaining items.
- Finally, it outputs whether the stack is empty.

TASK-2:

Prompt:

Implement a Queue with enqueue(), dequeue(), and is_empty() methods using Python lists.

Code and Output:

```
class Queue:
    def __init__(self):
        self.items = []

    def enqueue(self, item):
        """Add an item to the end of the queue."""
        self.items.append(item)

    def dequeue(self):
        """Remove and return the item from the front of the queue.

        Raises:
            IndexError: if the queue is empty.
        """
        if self.is_empty():
            raise IndexError("dequeue from an empty queue")
        return self.items.pop(0)

    def is_empty(self):
        """Return True if the queue is empty, otherwise False."""
        return len(self.items) == 0

# Example usage:
if __name__ == "__main__":
    q = Queue()
    print("Queue is empty:", q.is_empty()) # True
    q.enqueue(1)
    q.enqueue(2)
    q.enqueue(3)
    print("Queue is empty:", q.is_empty()) # False

    print("Dequeue:", q.dequeue()) # 1
    print("Dequeue:", q.dequeue()) # 2
    print("Dequeue:", q.dequeue()) # 3
    try:
        q.dequeue() # This should raise an error
    except IndexError as e:
        print("Error:", e)
```

Prompt:

Review performance and suggest a more efficient implementation (using collections.deque).

Code and Output:

```
from collections import deque

class Queue:
    def __init__(self):
        self.items = deque()

    def enqueue(self, item):
        """Add an item to the end of the queue."""
        self.items.append(item)

    def dequeue(self):
        """Remove and return the item from the front of the queue.

        Raises:
            IndexError: if the queue is empty.
        """
        if self.is_empty():
            raise IndexError("dequeue from an empty queue")
        return self.items.popleft()

    def is_empty(self):
        """Return True if the queue is empty, otherwise False."""
        return len(self.items) == 0

# Example usage:
if __name__ == "__main__":
    q = Queue()
    print("Queue is empty:", q.is_empty()) # True
    q.enqueue(1)
    q.enqueue(2)
    q.enqueue(3)
    print("Queue is empty:", q.is_empty()) # False

    print("Dequeue:", q.dequeue()) # 1
    print("Dequeue:", q.dequeue()) # 2
    print("Dequeue:", q.dequeue()) # 3
    try:
        q.dequeue() # This should raise an error
    except IndexError as e:
        print("Error:", e)
```

Code Explanation:

The code implements a Queue data structure using the efficient `collections.deque` from Python's standard library. Here's a breakdown:

- **Import Statement:**

The code imports `deque` from the `collections` module. `deque` (double-ended queue) allows fast appending and popping from both ends ($O(1)$ time complexity).

- **Class Definition (`Queue`):**

- `__init__(self)`: Initializes the queue by creating an empty `deque` instance and assigns it to the attribute `items`.
- `enqueue(self, item)`: Adds an element to the end of the queue using the `append()` method.
- `dequeue(self)`:
 - First checks if the queue is empty by calling `is_empty()`.
 - If empty, raises an `IndexError` to indicate that a removal was attempted on an empty queue.
 - If not empty, it uses the `popleft()` method of `deque` to remove and return the front item.
- `is_empty(self)`: Returns `True` if there are no elements in the queue (i.e., its length is 0), otherwise returns `False`.

- **Example Usage (under `if __name__ == "__main__":`)**

This block demonstrates how the `Queue` can be used:

- A new `Queue` instance is created.
- It first checks and prints whether the queue is empty.
- It enqueues three values (1, 2, and 3) and prints if the queue is still empty after these operations.
- It dequeues three items, printing each one to show the order (FIFO) functionality.
- Finally, an attempt to dequeue from the empty queue is made, which results in an `IndexError` that is caught and printed.

This implementation ensures that both enqueue and dequeue operations are efficient, with $O(1)$ performance, making it preferable over a list-based implementation that would have $O(n)$ for dequeuing.

TASK-3:

Prompt:

Implement a Singly Linked List with operations: `insert_at_end()`, `delete_value()`, and `traverse()`. Start with a simple class-based implementation (`Node`, `LinkedList`). Generate inline comments explaining pointer updates (which are non-trivial) also suggest test cases to validate all operations.

Code and Output:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None # Pointer to the next node in the list

class LinkedList:
    def __init__(self):
        self.head = None # Start with an empty list

    def insert_at_end(self, data):
        """Insert a new node with the provided data at the end of the list."""
        new_node = Node(data)
        # If list is empty, new node becomes the head
        if self.head is None:
            self.head = new_node
            # Pointer update: head now points to new_node
            return
        # Otherwise, traverse to the end of the list
        current = self.head
        while current.next:
            current = current.next
        # Pointer update: set the last node's next to the new node
        current.next = new_node

    def delete_value(self, value):
        """Delete the first node with a matching value."""
        current = self.head
        prev = None

        # If list is empty, nothing to delete
        if current is None:
            return

        # Check if the head node is the one to be deleted
        if current.data == value:
            # Pointer update: set head to the next node
            self.head = current.next
            return

        # Traverse looking for the value
        while current and current.data != value:
            prev = current
            current = current.next

        # If we found the value, update the pointer of the previous node
        if current:
            # Pointer update: bypass the current node
```

```

        current.next = new_node

def delete_value(self, value):
    """Delete the first node with a matching value."""
    current = self.head
    prev = None

    # If list is empty, nothing to delete
    if current is None:
        return

    # Check if the head node is the one to be deleted
    if current.data == value:
        # Pointer update: set head to the next node
        self.head = current.next
        return

    # Traverse looking for the value
    while current and current.data != value:
        prev = current
        current = current.next

    # If we found the value, update the pointer of the previous node
    if current:
        # Pointer update: bypass the current node
        prev.next = current.next

def traverse(self):
    """Traverse the list and print each node's data."""
    current = self.head
    while current:
        print(current.data, end=" -> " if current.next else "\n")
        # Update pointer to next node
        current = current.next

# Test cases to validate the operations
if __name__ == "__main__":
    ll = LinkedList()

    # Test insert_at_end
    print("Insertion test:")
    ll.insert_at_end(10)
    ll.insert_at_end(20)
    ll.insert_at_end(30)
    ll.traverse() # Expected output: 10 -> 20 -> 30

```

```

    # Test delete_value for a middle element
    print("After deleting 20:")
    ll.delete_value(20)
    ll.traverse() # Expected output: 10 -> 30

    # Test delete_value for the head
    print("After deleting 10 (head):")
    ll.delete_value(10)
    ll.traverse() # Expected output: 30

    # Test deletion for a non-existent value
    print("Attempt deletion of non-existent value 40:")
    ll.delete_value(40)
    ll.traverse() # Expected output: 30

    # Test deletion for the last element to get an empty list
    print("After deleting 30:")
    ll.delete_value(30)
    ll.traverse() # Expected output: (nothing/empty output)

```

Code Explanation:

Below is an explanation of the singly linked list code in the file:

1. Node Class

- The `Node` class represents each element in the linked list.
- It has two attributes:
 - `data`: Holds the value stored in the node.
 - `next`: Points to the next node in the list (initialized to `None`).

2. LinkedList Class

- The `LinkedList` class manages the nodes using the `head` pointer, which refers to the first node in the list (or `None` if the list is empty).
- `insert_at_end(data)`:
 - Creates a new node with the provided data.
 - If the list is empty (i.e., the `head` is `None`), the new node becomes the head.
 - Otherwise, it traverses to the end of the list (by following the `next` pointers) and then links the last node's `next` pointer to the new node.
- `delete_value(value)`:
 - Searches for the first node whose `data` matches the provided value.
 - There are two special cases:
 - If the list is empty, the method returns immediately.
 - If the matching node is the head, it updates the `head` pointer to the next node (effectively removing the first node).
 - If the matching node is found somewhere in the middle or tail of the list, the method updates the `next` pointer of the previous node to skip over the deleted node, effectively removing it from the chain.
- `traverse()`:
 - Starts at the `head` node and iterates through each node using the `next` pointer.
 - Prints each node's data, adding an arrow `"->"` between nodes to visually represent the linkage.

3. Test Cases (Main Block)

- The `if __name__ == "__main__":` block contains sample test cases that validate each operation:
 - **Insertion Test:**
 - Inserts nodes with values 10, 20, and 30, then outputs the list: expected output is "10 -> 20 -> 30".
 - **Delete Middle Value:**
 - Deletes the value 20 and verifies that the list becomes "10 -> 30".
 - **Delete Head:**
 - Deletes the head node (value 10) and checks that the remaining list is "30".
 - **Non-existent Value:**
 - Attempts to delete a value (40) that isn't in the list to ensure nothing changes.
 - **Delete Last Element:**
 - Deletes the remaining node (value 30) and verifies that the list is empty.

Each pointer update is carefully handled to maintain the integrity of the linked list structure, ensuring that when a node is inserted or removed, all links (i.e., the `next` pointers) are correctly updated.

TASK-4:

Prompt:

```
class Node:
```

```
    def __init__(self, key):
```

```
        self.key = key
```

```
        self.left = None
```

```
        self.right = None
```

```
class BST:
```

```
    def __init__(self):
```

```
        self.root = None
```

Here, is a partially written Node and BST class. Implement a Binary Search Tree with methods for insert(), search(), and inorder_traversal() and complete missing methods and add docstrings. Test with a list of integers and compare outputs of search() for present vs absent elements.

lab 11.py > ...

```
1 class Node:
2     """
3     A Node in the Binary Search Tree.
4
5     Attributes:
6         key: The key or value stored in the node.
7         left: Reference to the left child node.
8         right: Reference to the right child node.
9     """
10    def __init__(self, key):
11        self.key = key
12        self.left = None
13        self.right = None
14
15    class BST:
16        """
17        A simple Binary Search Tree (BST) implementation.
18
19        Methods:
20            insert(key): Inserts a key into the BST.
21            search(key): Searches for a key in the BST; returns True if found.
22            inorder_traversal(): Returns a list of keys representing an inorder traversal.
23        """
24        def __init__(self):
25            self.root = None
26
27        def insert(self, key):
28            """
29            Insert a key into the BST.
30
31            Args:
32                key: The key to be inserted.
33            """
34            if self.root is None:
35                self.root = Node(key)
36            else:
37                self._insert(self.root, key)
38
39        def _insert(self, node, key):
40            """
41            Helper method for insert() that recursively finds the position for the new key.
42
43            Args:
44                node: The current node in the BST.
45                key: The key to be inserted.
46            """
47            if key < node.key:
48                if node.left is None:
49                    node.left = Node(key)
50                else:
51                    self._insert(node.left, key)
52            elif key > node.key:
53                if node.right is None:
54                    node.right = Node(key)
55                else:
```

```

56         self._insert(node.right, key)
57     # Duplicates are not inserted in the BST.
58
59     def search(self, key):
60         """
61         Search for a key in the BST.
62
63         Args:
64             key: The key to search for.
65
66         Returns:
67             True if the key is found, False otherwise.
68         """
69         return self._search(self.root, key)
70
71     def _search(self, node, key):
72         """
73         Helper method for search() that recursively looks for a key.
74
75         Args:
76             node: The current node in the BST.
77             key: The key to search for.
78
79         Returns:
80             True if the key is found, False otherwise.
81         """
82         if node is None:
83             return False
84         if key == node.key:
85             return True
86         elif key < node.key:
87             return self._search(node.left, key)
88         else:
89             return self._search(node.right, key)
90
91     def inorder_traversal(self):
92         """
93         Perform an inorder traversal of the BST.
94
95         Returns:
96             A list of keys in the BST in sorted order.
97         """
98         result = []
99         self._inorder_traversal(self.root, result)
100         return result
101
102     def _inorder_traversal(self, node, result):
103         """
104         Helper method for inorder_traversal() that recursively appends keys.
105
106         Args:
107             node: The current node in the BST.
108             result: The list accumulating the keys.

```

```

109         """
110         if node:
111             self._inorder_traversal(node.left, result)
112             result.append(node.key)
113             self._inorder_traversal(node.right, result)
114
115     if __name__ == "__main__":
116         # Test the BST with a list of integers
117         test_values = [50, 30, 70, 20, 40, 60, 80]
118         bst = BST()
119         for val in test_values:
120             bst.insert(val)
121
122         print("Inorder Traversal of BST:")
123         print(bst.inorder_traversal())
124
125         # Compare search results for present and absent elements
126         present_key = 60
127         absent_key = 90
128         print(f"Search for present key ({present_key}): {bst.search(present_key)}")
129         print(f"Search for absent key ({absent_key}): {bst.search(absent_key)}")

```

```

Inorder Traversal of BST:
[20, 30, 40, 50, 60, 70, 80]
Search for present key (60): True
Search for absent key (90): False

```

Code Explanation:

The code implements a simple Binary Search Tree (BST) with the following components:

1. Node Class

- **Purpose:** Represents a single node in the BST.
- **Attributes:**
 - `key`: The value stored in the node.
 - `left`: A reference to the left child (smaller key).
 - `right`: A reference to the right child (larger key).

2. BST Class

- **Purpose:** Provides the structure and operations for the binary search tree.
- **Methods:**
 - `insert(key)`: Inserts a new key into the BST. If the tree is empty, the key becomes the root. Otherwise, it finds the correct position using the helper method `_insert(node, key)` based on the BST property (left child keys are less than the parent; right child keys are greater).
 - `_insert(node, key)`: A recursive helper that places the new key in its correct position.
 - `search(key)`: Returns `True` if the key exists in the BST, and `False` otherwise. It relies on the recursive helper `_search(node, key)`.
 - `_search(node, key)`: A recursive helper that checks the current node and then traverses the left or right subtree depending on whether the searched key is less than or greater than the current node's key.
 - `inorder_traversal()`: Returns a list of keys in the BST in sorted order. It uses the helper `_inorder_traversal(node, result)` to achieve an inorder traversal (left subtree, current node, then right subtree).
 - `_inorder_traversal(node, result)`: Recursively traverses the tree in inorder and appends keys to the result list.

3. Testing Code in Main Block

- A list of integers ([50, 30, 70, 20, 40, 60, 80]) is inserted into the BST.
- The inorder traversal is printed, showing the keys in sorted order.
- The `search()` method is tested with:
 - A key that exists (60), which should return `True`.
 - A key that does not exist (90), which should return `False`.

Each component is equipped with a docstring that explains its purpose and arguments, making the code easier to understand and maintain.

TASK-5:

Prompt:

Implement a Graph using an adjacency list, with traversal methods BFS() and DFS(). Start with an adjacency list dictionary. Generate BFS and DFS implementations with inline comments.

Code and Output:

```
lab 11.py > ...
1 class Graph:
2     def __init__(self, adjacency_list=None):
3         # Initialize the graph with an adjacency list; if none provided, start with an empty dict.
4         if adjacency_list is None:
5             adjacency_list = {}
6         self.adjacency_list = adjacency_list
7
8     def add_edge(self, src, dest):
9         # Add an edge from src to dest. This method can be extended for an undirected graph.
10        if src not in self.adjacency_list:
11            self.adjacency_list[src] = []
12        if dest not in self.adjacency_list:
13            self.adjacency_list[dest] = [] # Ensure the destination exists in the graph.
14        self.adjacency_list[src].append(dest)
15
16    def bfs(self, start):
17        # Perform Breadth-First Search starting from the given node.
18        visited = set() # A set to keep track of visited nodes.
19        queue = [] # Initialize the BFS queue.
20        traversal = [] # List to store the order of traversal.
21
22        queue.append(start) # Enqueue the starting node.
23        visited.add(start) # Mark the starting node as visited.
24
25        while queue: # Loop until the queue is empty.
26            node = queue.pop(0) # Dequeue the first node.
27            traversal.append(node) # Process the node.
28            # Enqueue all unvisited adjacent nodes.
29            for neighbor in self.adjacency_list.get(node, []):
30                if neighbor not in visited:
31                    visited.add(neighbor)
32                    queue.append(neighbor)
33        return traversal
34
35    def dfs(self, start):
36        # Perform Depth-First Search starting from the given node using recursion.
37        visited = set() # A set to keep track of visited nodes.
38        traversal = [] # List to store the order of traversal.
39
40        def _dfs_recursive(node):
41            visited.add(node) # Mark the node as visited.
42            traversal.append(node) # Process the node.
43            # Recursively visit all unvisited adjacent nodes.
44            for neighbor in self.adjacency_list.get(node, []):
45                if neighbor not in visited:
46                    _dfs_recursive(neighbor)
47
48        _dfs_recursive(start) # Begin the DFS from the start node.
49        return traversal
50
51 if __name__ == '__main__':
52     # Example usage:
53     # Define the graph using an adjacency list dictionary.
54     graph_data = {
55         'A': ['B', 'C'],
```

```

56         'B': ['D', 'E'],
57         'C': ['F'],
58         'D': [],
59         'E': ['F'],
60         'F': []
61     }
62
63     # Create a Graph instance with the provided adjacency list.
64     graph = Graph(graph_data)
65
66     # Print the traversal order using BFS starting from node 'A'.
67     print("BFS Traversal starting from A:", graph.bfs('A'))
68
69     # Print the traversal order using DFS starting from node 'A'.
70     print("DFS Traversal starting from A:", graph.dfs('A'))

```

```

BFS Traversal starting from A: ['A', 'B', 'C', 'D', 'E', 'F']
DFS Traversal starting from A: ['A', 'B', 'D', 'E', 'F', 'C']

```

Example of Iterative DFS:

```

def iterative_dfs(graph, start):
    visited = set() # Keep track of visited nodes.
    stack = [start] # Stack for DFS, initialized with the start node.
    traversal = [] # List to record the order of nodes visited.

    while stack:
        node = stack.pop() # Pop the last inserted element.
        if node not in visited:
            visited.add(node)
            traversal.append(node)
            # Push neighbors onto the stack in reversed order to maintain order.
            for neighbor in reversed(graph.get(node, [])):
                if neighbor not in visited:
                    stack.append(neighbor)
    return traversal

# Example usage:
graph_data = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}

print("Iterative DFS Traversal starting from A:", iterative_dfs(graph_data, 'A'))

```

Key Takeaways

- **Recursive DFS** leverages the programming language's own call stack, which is simpler but limited in depth.
- **Iterative DFS** uses an explicit stack to control the depth and prevent stack overflow, making it more robust for large graphs.

Code Explanation:

Code Explanation

- **Graph Class Initialization**
 - The `Graph` class uses an adjacency list (a dictionary) to represent the graph. If no list is provided, it initializes with an empty dictionary.
- **add_edge Method**
 - This method adds a directed edge from `src` to `dest`.
 - It checks if the source or destination nodes exist; if not, they are added as keys with empty lists, then it appends `dest` to the list for `src`.
- **bfs Method (Breadth-First Search)**
 - Starts by initializing a `visited` set to keep track of nodes that have been visited.
 - A list named `queue` implements the FIFO behavior. It starts from the `start` node, marking it as visited.
 - The loop dequeues a node, stores it in the `traversal` list, then enqueues its unvisited neighbors. The process continues until the queue is empty.
- **dfs Method (Depth-First Search)**
 - DFS is implemented recursively using an inner function `_dfs_recursive`.
 - It maintains a `visited` set and builds a `traversal` list by:
 - Marking the current node as visited.
 - Appending it to the traversal order.
 - Recursively processing each unvisited neighbor.
 - The recursion follows the call stack to explore as far as possible from each starting node before backtracking.

Recursive vs Iterative DFS

- **Recursive DFS**
 - **Implementation:** Uses function calls to maintain the state (call stack).
 - **Advantages:**
 - Simpler and more intuitive to implement.
 - Code is generally shorter and clearer.
 - **Disadvantages:**
 - Limited by the language's recursion stack size (risk of stack overflow on very deep graphs).
 - Harder to manage for graphs with very deep recursion or cycles without proper checks.
- **Iterative DFS**
 - **Implementation:** Uses an explicit stack data structure to mimic the recursive behavior.
 - **Advantages:**
 - Can handle deeper graphs without risking a stack overflow.
 - Greater control over the stack usage.
 - **Disadvantages:**
 - Code can be more verbose and less intuitive.
 - Requires careful management of the stack and visited nodes.